

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 4

RAG PIPELINE: INGESTION

Lesson 4: RAG Pipeline: Ingestion

"I have always imagined that Paradise will be a kind of library."

— Jorge Luis Borges

The "Why": The Amnesiac Expert

In Lesson 3, we gave our agent hands. It can now act by using tools. We have a conversational agent that can perform a specific, pre-defined skill. Yet, it's still profoundly limited. It's like a skilled intern who has a calculator but no access to the company's files. It has no deep knowledge, no long-term memory.

Our agent suffers from two fundamental constraints:

1. **Amnesia:** It forgets everything as soon as a conversation ends.
2. **A Tiny Attention Span:** Its "working memory" is limited by the LLM's context window. You simply cannot paste an entire codebase into a prompt.

This leads to a core problem we, as architects, must solve. In any organization or system, there is a natural tendency for knowledge to become inaccessible, fragmented, or lost over time. We call this **Epistemic Decay**. The agent's limited context window is a harsh form of this decay.

How do we build an antidote? How do we give our agent a perfect, searchable, long-term memory?

The "What": The RAG Indexing Pipeline

The solution is **Retrieval-Augmented Generation (RAG)**. Instead of trying to stuff the entire library into the agent's head, we will build the library and teach the agent how to use its card catalog.

RAG is a two-phase process:

1. **Indexing (Offline):** We process our source documents (the codebase) one time to create a highly efficient, searchable knowledge base. **This is our entire focus for this lesson.**
2. **Retrieval & Generation (Online):** The agent uses this knowledge base in real-time to find relevant information and answer user questions. (This will be covered in Lesson 5).

Today, we are the librarians. We will build the library. Our indexing pipeline consists of four distinct steps: **Load -> Split -> Embed -> Store.**

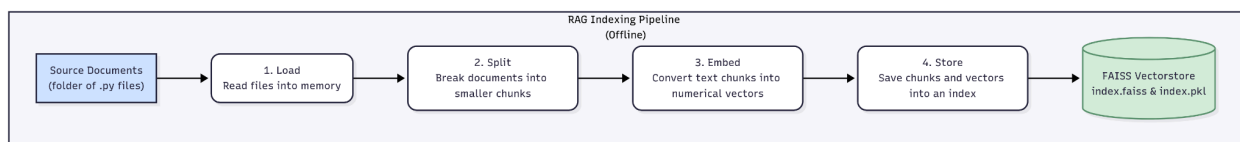


Figure 4-1: The RAG Indexing Pipeline. This is the offline, "librarian's work" that transforms raw source code from a folder of files into a structured, searchable FAISS vectorstore.

The "How" (The Build): Step-by-Step

Let's build the system that transforms our messy folder of code into a structured, intelligent knowledge base.

1. Evolve Your Project Structure

First, let's map out our project. We will add a directory to hold the source code we want to analyze and another to store the output of our new script.

```
/codebase-analyst/
├── tools/
│   ├── __init__.py
│   └── word_counter.py
├── source_code/           <-- NEW: To store code for analysis
├── vectorstore/           <-- NEW: To store our FAISS index
├── .env
└── .gitignore
```

```
|— 01_first_observable_call.py
|— 02_agent_state_machine.py
|— 03_agent_with_tools.py
|— 04_create_vectorstore.py      <-- NEW: Our indexing script
|— requirements.txt
```

2. Update Your Dependencies

Our RAG pipeline requires new, specialized libraries. Update your requirements.txt file to include them.

- faiss-cpu: The engine for our vector database. cpu means it runs on your computer's main processor.
- sentence-transformers: A library that provides powerful, open-source embedding models.
- unstructured: A library for loading and processing various document types.

Your requirements.txt should now look like this:

```
python-dotenv
langchain
langchain-openai
langsmith
langgraph
langchain_community
faiss-cpu
sentence-transformers
unstructured
langchain-huggingface
```

Now, run the installer from your activated (venv) terminal:

```
pip install -r requirements.txt
```

3. Prepare Your Data Corpus

1. Create the new directories:

```
mkdir source_code
mkdir vectorstore
```

2. Copy the code you've written so far into the source_code directory. This will be the "knowledge" our agent learns.
 - Copy 01_first_observable_call.py into source_code/.
 - Copy 02_agent_state_machine.py into source_code/.
 - Copy 03_agent_with_tools.py into source_code/.

4. Code Walkthrough: The Indexing Script

This script is a standalone utility. Its only job is to perform the Load, Split, Embed, and Store process.

- **File:** 04_create_vectorstore.py

Pristine Code Block

```
# 04_create_vectorstore.py

from langchain_community.document_loaders import DirectoryLoader,
TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

SOURCE_CODE_PATH = "./source_code"
VECTORSTORE_PATH = "./vectorstore"

def main():
    """
    This script creates a FAISS vectorstore from the source code
    documents.
```

```
It follows the Load -> Split -> Embed -> Store pipeline.
"""

print("--- Starting Vectorstore Creation ---")

# --- 1. LOAD ---
# Use a DirectoryLoader to ingest all .py files.
print(f"Loading documents from {SOURCE_CODE_PATH}...")
loader = DirectoryLoader(
    SOURCE_CODE_PATH,
    glob="**/*.py",          # Load only Python files
    loader_cls=TextLoader   # Use TextLoader for .py files
)
documents = loader.load()
print(f"Loaded {len(documents)} document(s).")

# --- 2. SPLIT ---
# Use a RecursiveCharacterTextSplitter for intelligent chunking.
print("Splitting documents into chunks...")
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
texts = text_splitter.split_documents(documents)
print(f"Split into {len(texts)} chunks.")

# --- 3. EMBED ---
# Select a high-quality, open-source embedding model that runs
locally.
print("Initializing embedding model...")
model_name = "sentence-transformers/all-MiniLM-L6-v2"
embeddings = HuggingFaceEmbeddings(model_name=model_name)

# --- 4. STORE ---
# Create the FAISS vectorstore from the chunks and embeddings.
print("Creating FAISS vectorstore...")
db = FAISS.from_documents(texts, embeddings)
db.save_local(VECTORSTORE_PATH)

print("--- Vectorstore Creation Complete ---")
print(f"Vectorstore saved at: {VECTORSTORE_PATH}")
```

```
if __name__ == "__main__":  
    main()
```

Deconstructing the Script

This standalone script is a pure data engineering pipeline. Let's break down each stage:

- **Load:** We use `DirectoryLoader` to recursively find all `.py` files in our `source_code` folder.
- **Split:** We use `RecursiveCharacterTextSplitter`, a smart splitter that tries to break up text along logical boundaries (like newlines and spaces). The `chunk_overlap` ensures that we don't lose context between chunks.
- **Embed:** We choose `all-MiniLM-L6-v2`, a very popular and high-performing model that runs entirely on your local machine (no API key needed for this part). The `HuggingFaceEmbeddings` class handles the complexity of downloading and running it.

Deeper Dive: How Local Embeddings Work

It's easy to treat `HuggingFaceEmbeddings` as a black box, but a professional builder knows their tools. When you run this code, it's not magic; it's a smart, efficient workflow.

The `HuggingFaceEmbeddings` class acts like a contractor's assistant. The first time you need a specific tool (the `all-MiniLM-L6-v2` model), the assistant drives to the hardware store (the Hugging Face Hub), buys the tool, and puts it in your truck (a local cache folder). You'll see this one-time purchase as a download bar in your terminal.

From then on, every time you need the tool, the assistant just grabs it from the truck. It's fast, free, and works even if you're offline.

This is a deliberate architectural decision. By embedding locally, we gain three non-negotiable advantages: it's **free** (saving on API costs), it's **private** (our sensitive source code never leaves our machine), and it's **reliable** (no network dependency).

- **Store:** The `FAISS.from_documents()` method is the powerhouse. It takes our text chunks, runs each one through the embedding model to get a vector, and then stores the chunk and its vector together in an efficient index. Finally, `.save_local()` writes this index to our disk.

The Integrated Run & Analysis

```
python 04_create_vectorstore.py
```

Expected Console Output:

The first time you run this, it will download the embedding model, which may take a minute or two.

```
--- Starting Vectorstore Creation ---
Loading documents from ./source_code...
Loaded 3 document(s).
Splitting documents into chunks...
Split into 4 chunks.
Initializing embedding model...
/path/to/venv/.../sentence_transformers/cross_encoder/CrossEncoder.py:11
: TqdmExperimentalWarning: Using `tqdm.autonotebook`.
... (Model download progress bars) ...
Creating FAISS vectorstore...
--- Vectorstore Creation Complete ---
Vectorstore saved at: ./vectorstore
```

 **Heads Up:** You will likely see several progress bars and `UserWarning` or `FutureWarning` messages. This is completely normal when using these libraries for the first time. As long as the script completes, you can safely ignore them.

Analysis: What Did We Just Create?

Navigate to your new vectorstore directory. You will see two files:

- **index.faiss:** This is the engine of our memory. It's a binary file containing a highly optimized numerical index of all the vectors. Think of it as the **"map of meaning"** itself.
- **index.pkl:** This is a Python pickle file that acts as the **"card catalog."** It stores the mapping between the vector IDs in index.faiss and the original text chunks.

You have successfully transformed your unstructured code into a structured, searchable knowledge base. You have built the library.

Summary & Key Takeaways

This lesson was our deep dive into the practical data engineering that powers modern AI. We've built the foundation for our agent's expertise.

Mental Models You've Gained:

- **Antidote to Epistemic Decay:** You now see RAG not just as a technique, but as a powerful architectural pattern to make knowledge persistent and accessible to an AI.
- **Indexing as an Offline Data Prep Step:** You understand that building a knowledge base is a distinct, one-time process, separate from the live agent's reasoning loop.
- **Embeddings as Semantic Coordinates:** You have an intuitive grasp of how any piece of text can be converted into a meaningful numerical representation for similarity searching.
- **Smart Chunking Matters:** You recognize that how you split your documents is a critical engineering decision that directly impacts the quality of your RAG system.

To make the idea of "semantic coordinates" more concrete, consider this visualization of a vector space. The embedding model's job is to place every piece of your text onto a high-dimensional version of this map. Our RAG system then works by finding the nearest neighbors on the map.

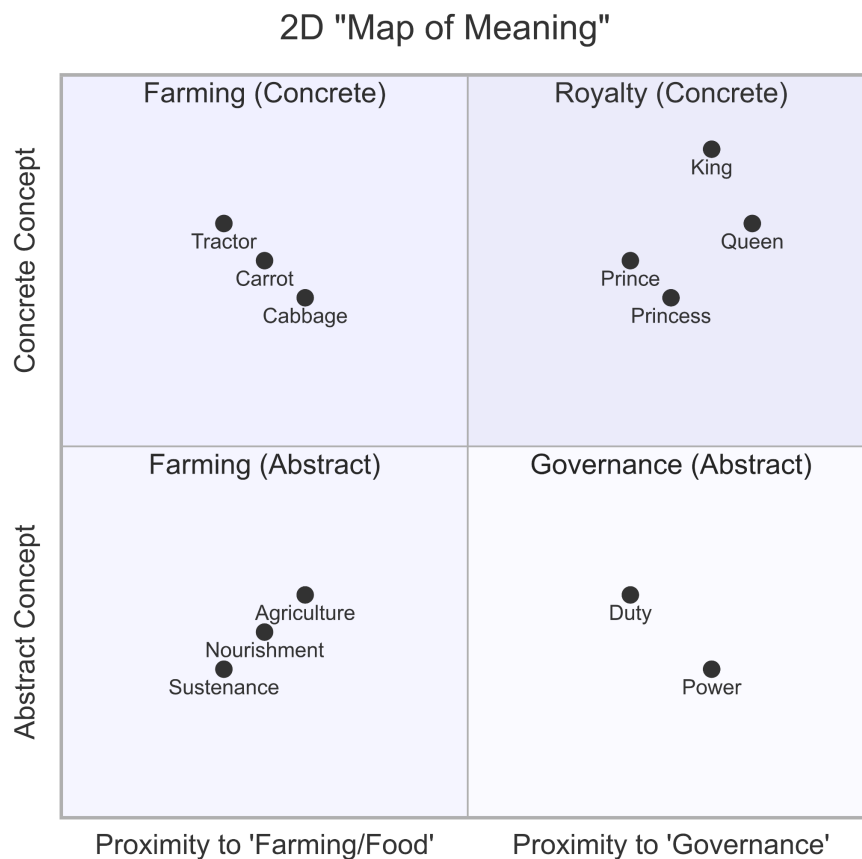


Figure 4-2: Visualizing Vector Space – The "Map of Meaning."

An embedding model plots concepts as coordinates. Semantically similar items (like "King" and "Queen," or "Tractor" and "Carrot") are clustered closely together. The distance between points represents their difference in meaning, which is the mathematical foundation of our semantic search.

This mental model of a "map of meaning" is one of the most powerful in all of AI engineering.

What's Next

Our agent now has a world-class library at its disposal. But it doesn't know the library exists, nor does it have a library card.

In Lesson 5, we will complete the RAG circuit. We will build a new **Retriever Tool** that allows our agent to query this vectorstore, giving it the superpower to read, understand, and answer questions about our codebase.